

ПЛАН ВЫПОЛНЕНИЯ ЗАПРОСА

План выполнения запроса в PostgreSQL — это детальный алгоритм, по которому СУБД действительно *выполняет* SQL. Он создается автоматически планировщиком (planner) и зависит от структуры запроса, статистики таблиц, индексов и допустимых стратегий. PostgreSQL может сгенерировать десятки/сотни альтернативных планов, и выберет самый дешевый по оценочной модели.

Что такое план выполнения запроса

SQL — **декларативный** язык: ты говоришь, *что* хочешь получить, а не *как*.

СУБД строит **программу** (query plan), которая:

1. Выбирает порядок операций (фильтрация, проекция, соединение)
2. Выбирает физические алгоритмы (Index Scan, Hash Join и т.д.)
3. Оценивает **стоимость** каждого варианта
4. Выбирает оптимальный (наименее затратный)

Этапы построения плана

1. **Парсинг (parser)**
 - Синтаксический анализ SQL → дерево операций
2. **Переписывание (rewriter)**
 - Разворачивание представлений, CTE, RULE-ов
3. **Планирование (planner)**
 - Перебор всех валидных стратегий
 - Оценка стоимости каждой (в "единицах стоимости")
 - Выбор самого "дешевого"
4. **Исполнение (executor)**
 - Пошаговое выполнение итогового плана

Как оценивается "стоимость"

Каждая операция (фильтр, JOIN, сортировка и т.д.) имеет:

- **startup cost** — стоимость начала операции (например, создания хеша)
- **total cost** — общая стоимость (включая все проходы)

Оценка включает:

- Число чтений/записей с диска
- Затраты на CPU
- Объем обрабатываемых строк
- Потребление памяти

Единицы абстрактны — это не миллисекунды, а **весовые коэффициенты** для сравнения вариантов.

Какие решения принимает планировщик

1. Физический алгоритм доступа

- Seq Scan — построчное сканирование
- Index Scan — использование B-дерева для точечного/диапазонного поиска
- Bitmap Index Scan — оптимизированный способ выборки **многих строк по индексу**, с последующим объединением и чтением данных по страницам

2. Порядок соединения таблиц

- В каком порядке делать JOIN
- Какая таблица внешняя, какая внутренняя

3. Алгоритм соединения (про это подробно в оптимизации)

- Nested Loop
- Hash Join
- Merge Join

4. Когда выполнять фильтры

- До соединения или после?
- Ранний WHERE = меньше данных = дешевле

5. Когда и как делать проекции

- До JOIN → меньше столбцов → быстрее
- После JOIN → дороже

Что помогает планировщику

1. Статистика ANALYZE

- Частота значений
- Количество строк
- Селективность условий

2. Индексы

- Дают альтернативные стратегии доступа

3. Ключи, ограничения

- Помогают исключить невозможные строки

4. LIMIT / OFFSET

- Можно завершить выполнение раньше → выгоднее Nested Loop

5. Настройки памяти

- work_mem, random_page_cost, seq_page_cost

EXPLAIN ANALYZE

Выведет:

- Какой план был выбран
- Сколько строк реально обработано
- Использовался ли индекс
- Где были затраты

Подробно про Физические алгоритмы доступа

Sequential Scan (Seq Scan)

- Чтение **всех страниц таблицы с диска**;
- Каждая строка проверяется вручную;
- **Не зависит от наличия индекса**;
- Используется при:
 - Отсутствии подходящих индексов;
 - Ожидании, что выберется **много строк** (низкая селективность);
 - Очень маленьких таблицах (где быстрее просканировать всё).
- Простой, работает всегда, эффективен для **маленьких таблиц** или **больших выборок**.
- Очень медленно при **выборке малого количества строк** из больших таблиц.

Index Scan

- Происходит **поиск в индексе** (обычно логарифмический).
- Для каждой строки: **индекс → heap (таблица) → visibility check**.
- Позволяет выполнять **диапазонные выборки**.
- Применимо для условий:
 - =, >, <, BETWEEN, LIKE 'abc%'
- Очень быстро на **точечный доступ** (id = 18), эффективно при **небольшом числе подходящих строк**
- При **многих подходящих строках** становится неэффективным, т.к. для каждой строки нужен отдельный **heap access** (чтение строки из таблицы).

Bitmap Index Scan + Bitmap Heap Scan

Используется при

- **много условий в WHERE**, например:
 - a = 10 OR b = 20
 - a IN (1,2,3,...)
- или при **выборке средней/высокой селективности** (например, 10-50% строк)
- Гораздо быстрее Index Scan, если **нужно много строк, объединяет индексы** (поддерживает OR, IN, сложные условия).
- Требуется **дополнительной памяти** под битовую карту, **не работает**, если нет индексированного поля.

Метод доступа	Принцип работы	Идеален для	Когда используется
Sequential Scan	Полный проход по таблице	Все строки, нет индекса	Много строк, нет/плохой индекс
Index Scan	Поиск по индексу → чтение строк	Точная/диапазонная выборка	Малое количество строк, =/<
Bitmap Index Scan	Множественные попадания → битовая карта	Много строк, сложные OR	Условий много, строки разбросаны